🗺️ ShelfMind — Hoja de Ruta de Desarrollo

Generado: 28/02/2026 — Basado en sesión de brainstorming

📋 Índice

1. Cómo leer este documento
2. Etapa 1 — Bot Worker: Mejoras de sesión
3. Etapa 2 — Base de datos: Tabla de bonos
4. Etapa 3 — API: Endpoints de bonos
5. Etapa 4 — Admin Panel: Configuración de bonos
6. Etapa 5 — Reportes: Timeline + Bonos + Excel
7. Resumen visual de dependencias
8. Reglas de negocio — Referencia rápida

Cómo leer este documento

Cada etapa tiene:

- **Qué es:** descripción simple de qué vamos a construir
- **Por qué en este orden:** razón por la que viene antes o después de otra etapa
- **Pasos concretos:** las tareas específicas, una por una
- **Archivos que se tocan:** exactamente qué archivos del proyecto se modifican
- **Resultado esperado:** cómo sabés que esa etapa está terminada

Las etapas están ordenadas por **dependencia**: no se puede hacer la 3 sin la 2, ni la 5 sin la 3. Siempre podés hacer la etapa que sigue cuando la anterior está terminada.

Etapa 1 — Bot Worker: Mejoras de sesión

Qué es

Mejorar cómo el bot maneja las fotos que mandan los vendedores, especialmente cuando pasa algo inesperado: mandan varias fotos a la vez, no responden, o mandan una foto nueva antes de terminar la anterior.

Por qué en este orden

Esta etapa es **completamente independiente** del resto. No toca la base de datos de bonos ni los reportes. Es el mejor lugar para arrancar porque los cambios están aislados en un solo archivo (`bot_worker.py`) y se pueden probar inmediatamente en Telegram.

Pasos concretos

Paso 1.1 — Cola de espera (foto nueva con sesión activa)

Hoy cuando un vendedor manda una foto nueva mientras tiene una carga sin terminar, el bot pisa la sesión anterior sin avisar. La solución es agregarle al bot una segunda "bandeja" llamada cola de espera.

- Agregar una variable nueva en memoria: `waiting_queue` (diccionario por usuario)
- Cuando llega una foto y ya hay una sesión activa para ese usuario:
  - Si la cola ya tiene 1 foto → responder con: *"Ya tenés una imagen esperando registro. Por favor, terminá primero la carga que está en curso antes de enviar más fotos."* y NO guardar la nueva foto
  - Si la cola está vacía → guardar la foto nueva en la cola y responder con: *"Recibí tu foto, pero primero necesito que termines de registrar la que estás cargando ahora. ¡Ya casi!"*
- Cuando termina una sesión (el vendedor apretó el botón de tipo PDV y se registró todo) → el bot revisa si hay algo en `waiting_queue`. Si hay una foto esperando, arranca automáticamente la sesión con esa foto y le dice: *"¡Listo! Ahora sí, ¿a qué cliente corresponde esta foto?"* respondiendo directamente a la imagen en espera

Paso 1.2 — Expiración de sesiones con aviso (15 minutos)

Hoy las sesiones mueren a los 10 minutos en silencio. Hay que:

- Cambiar el tiempo de expiración de 10 a 15 minutos
- Cuando el `cleanup_sessions_job` detecta una sesión expirada, en lugar de simplemente borrarla, debe enviar un mensaje al grupo antes de borrarla
  - Si tenía sesión activa sin completar: *" ⏰ Sesión finalizada por inactividad. Si querés cargar esa exhibición, enviá la foto nuevamente."*
  - Si además tenía una foto en la cola de espera: *" ⏰ También tenía una foto tuya esperando, pero la perdí por el tiempo. Enviá ambas fotos nuevamente cuando estés listo."*
- La foto en la cola de espera también se elimina cuando expira la sesión activa
- El mensaje de expiración debe ser una respuesta a la foto original del vendedor (para dar contexto)

Paso 1.3 — Aviso de demasiadas fotos

Si un vendedor intenta mandar una segunda foto cuando ya tiene una en la cola de espera:

- Responder con: *"📸 Estás enviando muchas fotos sin registrar. Por favor, esperá a que te guíe con las que ya tengo guardadas antes de enviar más."*
- No guardar la foto nueva, ignorarla completamente

Archivos que se tocan

- `CenterMind/bot_worker.py` — único archivo modificado

Resultado esperado

- Un vendedor puede mandar una foto, no responder 15 minutos, y recibir un aviso claro
- Un vendedor puede mandar foto nueva mientras tiene otra pendiente, y el bot lo guía en orden sin perder ninguna
- Nunca más una sesión muere en silencio

Etapa 2 — Base de datos: Tabla de bonos

Qué es

Agregar las tablas necesarias en la base de datos para poder guardar la configuración de bonos de cada distribuidora, mes a mes.

Por qué en este orden

La base de datos es el **cimiento**. La API (Etapa 3), el panel admin (Etapa 4) y los reportes (Etapa 5) todos necesitan leer y escribir estas tablas. Si no existen las tablas, nada de lo demás funciona. Siempre se construye la base antes que lo que la usa.

Pasos concretos

Paso 2.1 — Crear tabla `bonos_config`

Esta tabla guarda la configuración de bonos de cada distribuidora para cada mes. Una fila por distribuidora por mes.

Columnas:

- `id_config` — número automático, identificador único
- `id_distribuidor` — a qué distribuidora corresponde
- `anio` — el año (ej: 2026)
- `mes` — el mes del 1 al 12
- `umbral_puntos` — cantidad de puntos necesarios para el bono fijo (ej: 100)

- `monto_bono_fijo` — lo que cobra quien llega al umbral (ej: 200000)
- `monto_por_punto` — lo que cobra por punto quien NO llega al umbral (ej: 500)
- `edicion_bloqueada` — 0 o 1, si el superadmin bloqueó al admin para editar este mes
- `creado_en` — fecha de creación del registro
- `modificado_en` — fecha de última modificación

Paso 2.2 — Crear tabla `bonos_ranking`

Esta tabla guarda los premios por puesto de ranking. Puede tener muchas filas por configuración (una por puesto premiado).

Columnas:

- `id_ranking` — número automático
- `id_config` — FK a `bonos_config`, a qué configuración mensual pertenece
- `puesto` — el número de puesto (1, 2, 3, ...)
- `monto_si_llega` — premio si ese puesto cumplió el umbral
- `monto_si_no_llega` — premio si ese puesto NO cumplió el umbral

Paso 2.3 — Modificar `setup_db.py`

Agregar el `CREATE TABLE IF NOT EXISTS` de ambas tablas nuevas al script de setup. Esto es importante porque si alguien instala el sistema desde cero, las tablas se crean automáticamente.

> ⚠️ **Importante:** Usar siempre `CREATE TABLE IF NOT EXISTS` para que el script sea seguro de correr sobre una base de datos existente sin romper nada.

Archivos que se tocan

- `CenterMind/base_datos/setup_db.py` — agregar las dos tablas nuevas

Resultado esperado

- Las dos tablas existen en la base de datos
- Se puede verificar con DB Browser abriendo `centermind.db` y confirmando que `bonos_config` y `bonos_ranking` aparecen en la lista de tablas

---

Etapa 3 — API: Endpoints de bonos

Qué es

Agregar a la API (`api.py`) los "puentes" de comunicación que permiten leer y escribir la configuración de bonos, y calcular cuánto le corresponde cobrar a cada vendedor.

Por qué en este orden

La API es el **intermediario** entre la base de datos y las pantallas. Sin los endpoints, el panel admin no puede guardar la configuración, y reportes no puede mostrar los cálculos. Las tablas ya existen (Etapa 2), ahora construimos las puertas para acceder a ellas.

Pasos concretos

Paso 3.1 — Endpoint: Leer configuración del mes

`GET /bonos/config/{id_distribuidor}?anio=2026&mes=2`

Devuelve la configuración de bonos del mes pedido. Si no existe configuración para ese mes, devuelve vacío. Incluye tanto los datos de `bonos_config` como la lista completa de puestos de `bonos_ranking`.

Paso 3.2 — Endpoint: Guardar configuración del mes

`POST /bonos/config`

Recibe toda la configuración (umbral, montos, lista de puestos de ranking) y la guarda. Si ya existe configuración para ese mes, la sobreescribe. Antes de guardar, verifica que el que está pidiendo guardar sea el admin de esa distribuidora O un superadmin, y que la edición no esté bloqueada (salvo que sea superadmin, que puede siempre).

Paso 3.3 — Endpoint: Bloquear/desbloquear edición

`POST /bonos/config/{id_config}/bloqueo`

Solo accesible para superadmin. Cambia el campo `edicion_bloqueada` entre 0 y 1.

Paso 3.4 — Endpoint: Calcular liquidación del mes

`GET /bonos/liquidacion/{id_distribuidor}?anio=2026&mes=2`

Este es el endpoint más importante. Hace todo el cálculo:

1. Lee la configuración del mes de `bonos_config` y `bonos_ranking`
2. Suma los puntos de cada vendedor (aprobadas × 1 + destacadas × 2) del período
3. Determina si cada vendedor llegó o no al umbral
4. Calcula el monto base de cada uno (bono fijo o monto × puntos)
5. Construye el ranking ordenado por puntos, desempatando por aprobadas

6. Agrega el premio de ranking a quien corresponda
7. Devuelve todo: puntos por vendedor, posición en ranking, monto base, premio ranking, **total a cobrar**

Nota: si el mes no cerró todavía, el cálculo se hace con los datos que hay hasta hoy (proyección).

Paso 3.5 — Endpoint: Detalle diario del vendedor

`GET /bonos/detalle/{id_distribuidor}/{id_vendedor}?anio=2026&mes=2`

Devuelve día a día cuántas aprobadas, destacadas y rechazadas tuvo el vendedor, y cuántos puntos acumulados llevaba hasta ese día. Esto alimenta el gráfico de "el viaje" en el reporte.

Archivos que se tocan

- `CenterMind/api.py` — agregar los 5 endpoints nuevos

Resultado esperado

- Se puede llamar a `GET /bonos/liquidacion/1?anio=2026&mes=2` y obtener un JSON con el cálculo completo de cuánto cobra cada vendedor
- El cálculo es correcto: quien llega al umbral cobra el bono fijo + su premio de ranking premium, quien no llega cobra por punto + su premio de ranking reducido

---

Etapa 4 — Admin Panel: Configuración de bonos

Qué es

Agregar una sección nueva en el panel de administración (`3a_Admin_pc.py`) donde el admin de cada distribuidora pueda configurar los bonos del mes: el umbral, los montos, y los premios por puesto de ranking.

Por qué en este orden

Necesita que la API ya esté lista (Etapa 3) porque la pantalla habla con la API para guardar y leer los datos. Sin los endpoints, la pantalla no puede hacer nada útil.

Pasos concretos

Paso 4.1 — Nuevo tab "💰 BONOS" en el panel admin

Agregar un tab nuevo en la pantalla de admin, visible para el admin del distribuidor y para el superadmin.

Paso 4.2 — Selector de mes

Al tope del tab, un selector de mes y año. Al cambiar la selección, carga la configuración guardada de ese mes (si existe). Si no existe, muestra el formulario en blanco listo para completar.

Paso 4.3 — Formulario de configuración base

Campos editables:

- Umbral de puntos para el bono (número entero)
- Monto del bono fijo al llegar al umbral (en pesos)
- Monto por punto para quien no llega (en pesos)

Si la edición está bloqueada por el superadmin, los campos se muestran en modo lectura (no editables), con un cartel que dice *"La configuración de este mes fue bloqueada por el administrador del sistema."*

Paso 4.4 — Tabla de premios por ranking

Una tabla editable donde el admin puede agregar y quitar filas. Cada fila tiene:

- Puesto (número)
- Premio si llega al umbral (en pesos)
- Premio si NO llega al umbral (en pesos)

Botón para agregar fila, botón de eliminar por fila.

Paso 4.5 — Botón guardar y vista previa

- Botón "Guardar configuración" que llama al endpoint de la Etapa 3
- Debajo del formulario, una sección de vista previa que muestra cómo se vería la configuración si el mes cerrara hoy: tabla con cada vendedor, sus puntos, si llegó o no, y el monto calculado. Esto es solo informativo, no se puede modificar desde ahí.

Paso 4.6 — Control de bloqueo (solo superadmin)

Si el usuario es superadmin, aparece un botón extra: "🔒 Bloquear edición" / "🔓 Desbloquear edición". Al hacer clic, llama al endpoint de bloqueo (Etapa 3.3) y el estado cambia visualmente.

Archivos que se tocan

- `CenterMind/StreamLitApp/pages/3a_Admin_pc.py` — agregar el tab y toda la lógica

Resultado esperado

- El admin puede entrar al panel, elegir el mes de Marzo 2026, poner el umbral en 100 puntos, el bono fijo en $200.000, $500 por punto, y una tabla con premios para los primeros 5 puestos, y guardar todo
- Si el superadmin bloquea esa configuración, el admin ya no puede editarla
- La vista previa muestra el cálculo en tiempo real con los vendedores actuales

Etapa 5 — Reportes: Timeline + Bonos + Excel

Qué es

Mejorar la sección de reportes en tres frentes: arreglar el gráfico de timeline (que hoy es feo), agregar el reporte completo de bonos con el viaje y el destino de cada vendedor, y solucionar el error de openpyxl para que la descarga de Excel funcione.

Por qué en este orden

Es la última etapa porque necesita todo lo anterior: los datos de bonos vienen de los endpoints de la Etapa 3, y el reporte de bonos solo tiene sentido si la configuración ya existe (Etapa 4). El fix de openpyxl lo hacemos acá porque está en el mismo archivo y tiene sentido resolverlo todo junto.

Pasos concretos

Paso 5.1 — Fix de openpyxl

El error *"Instalá openpyxl para descargar en Excel"* aparece porque la librería no está instalada en el servidor. La solución es:

1. Correr `pip install openpyxl` en la consola del servidor donde corre Streamlit
2. Agregar `openpyxl` al archivo `requirements.txt` del proyecto para que quede documentado y cualquier instalación futura lo incluya automáticamente
3. Verificar que el botón de descarga de Excel funciona en `4a_Reportes_pc.py`

Paso 5.2 — Timeline mejorado

Reemplazar el timeline actual por un gráfico de línea que muestre la actividad día a día. Características:

- Eje X: días del período seleccionado (solo fechas, sin horas)
- Eje Y: cantidad de exhibiciones
- 2 líneas de colores distintos: Aprobadas (verde) y Rechazadas (rojo)
- Filtro por vendedor: un selector arriba del gráfico. Por defecto muestra "Todo el equipo" (suma de todos). Al seleccionar un vendedor, muestra solo sus datos
- Al hacer clic en un punto del gráfico, muestra un panel lateral o un popover con el detalle de ese día: lista de exhibiciones con su estado, el vendedor que las hizo, y el cliente

Paso 5.3 — Sección nueva: Reporte de Bonos

Agregar una sección nueva en `4a_Reportes_pc.py` accesible solo para admin y superadmin.

Sub-sección A: Resumen ejecutivo del mes

Esta es la primera parte, el "destino". Una página que muestra:

- Encabezado: nombre de la distribuidora, mes y año
- Bloque de condiciones del mes: umbral, bono fijo, monto por punto, tabla de premios de ranking (tal como fue configurado)
- Tabla de ranking del mes con columnas: Puesto | Vendedor | Aprobadas | Destacadas | Puntos | ¿Llegó al umbral? | Monto base | Premio ranking | **TOTAL A COBRAR**
- Si el mes no cerró, un cartel claro que dice *"Proyección al [fecha de hoy] — El mes sigue en curso"*
- Botón para descargar el reporte completo en Excel

Sub-sección B: Detalle por vendedor — "El viaje"

Debajo del resumen, una sección expandible por vendedor (o en tabs). Para cada vendedor muestra:

- Nombre, mes, distribuidora
- Tabla día a día: Fecha | Aprobadas del día | Destacadas del día | Rechazadas del día | Puntos del día | Puntos acumulados
- Gráfico de línea de puntos acumulados a lo largo del mes, con una línea horizontal punteada que marca el umbral. Visualmente se ve claramente si el vendedor cruzó el umbral y en qué momento
- Bloque de cálculo escrito, listo para imprimir o copiar:

```
LIQUIDACIÓN — [Nombre Vendedor] — [Mes Año]
──────────────────────────────────────────────────────────────

Exhibiciones aprobadas:  XX  (XX puntos)
Exhibiciones destacadas: XX  (XX puntos)
Exhibiciones rechazadas: XX  (0 puntos)
                         ──────────────
Total de puntos:            XX puntos

Umbral del mes:             100 puntos
Estado:                     ✅ ALCANZADO / ❌ NO ALCANZADO

Cálculo:
  [Si llegó]  Bono fijo por 100+ puntos:    $200.000
  [Si no llegó]  XX puntos × $500/punto:    $XX.XXX

Posición en ranking:        #X
Premio ranking:             $XX.XXX

──────────────────────────────────────────────────────────────
TOTAL A COBRAR:             $XXX.XXX
──────────────────────────────────────────────────────────────
```

Paso 5.4 — Exportación a Excel del reporte de bonos

El Excel descargable debe tener múltiples hojas:

- Hoja 1 "Resumen": la tabla del ranking con todos los montos
- Hoja 2 "Condiciones": la configuración del mes (umbral, montos, premios de ranking)
- Una hoja por vendedor: con la tabla día a día de su actividad y su cálculo final

Archivos que se tocan

- `CenterMind/StreamLitApp/pages/4a_Reportes_pc.py` — timeline + sección bonos + excel
- `requirements.txt` — agregar openpyxl

Resultado esperado

- El botón de descarga Excel funciona sin error
- El timeline muestra dos líneas limpias por día, filtrable por vendedor, con detalle al hacer clic
- El reporte de bonos muestra el resumen ejecutivo y el viaje de cada vendedor
- El Excel descargable tiene todas las hojas con el cálculo listo para presentar

---

Resumen visual de dependencias

```
Etapa 1 — Bot Worker
  ↓ (independiente, se puede hacer en paralelo con 2)

Etapa 2 — Base de datos
  ↓

Etapa 3 — API (endpoints de bonos)
  ↓
  ├── Etapa 4 — Admin Panel (configurar bonos)
  └── Etapa 5 — Reportes (leer y mostrar bonos)
```

Las Etapas 1 y 2 se pueden trabajar en paralelo.
La Etapa 3 desbloquea tanto la 4 como la 5.
La Etapa 5 puede arrancarse con el fix de openpyxl y el timeline (Pasos 5.1 y 5.2) antes de tener lista la Etapa 3.

Reglas de negocio — Referencia rápida

Esta sección es un resumen de las decisiones tomadas en el brainstorming. Sirve de referencia cuando estés codificando para no tener que releer todo.

Sistema de puntos

- Exhibición Aprobada = 1 punto
- Exhibición Destacada = 2 puntos
- Exhibición Rechazada = 0 puntos (no resta)

Lógica de bonos

```
SI puntos_del_mes >= umbral_del_mes:
  monto_base = monto_bono_fijo
SINO:
  monto_base = puntos_del_mes × monto_por_punto

premio_ranking = monto según puesto en ranking
                 (usando la columna correcta: si llegó o no llegó)
```

```
total = monto_base + premio_ranking
```

Desempate en ranking

Si dos vendedores tienen los mismos puntos, gana el que tenga más exhibiciones Aprobadas.

Período

- Los bonos son mensuales
- El cobro es a mes vencido
- El reporte puede mostrar proyección del mes en curso

Permisos de configuración

- Admin del distribuidor: puede configurar bonos de su distribuidora, salvo que esté bloqueado
- Superadmin: puede configurar cualquier distribuidora y puede bloquear/desbloquear la edición

Cola del bot

- Máximo 1 foto en espera por usuario
- Si manda una tercera foto, se rechaza con aviso
- A los 15 minutos expira todo (sesión activa + cola), siempre con aviso
- Al terminar la sesión activa, se reactiva automáticamente la foto en espera

---